GUÍA DE LABORATORIO

Laboratorio de Experiencias Web Inmersivas — Framer Motion

Semana 9 | Proyecto integrador: Plataforma Web de Experiencias Interactivas

Producto articulador: una tarjeta de experiencia tipada con TypeScript y animada con Framer Motion.

1. Datos generales

Elemento	Descripción
Semana	**9**
Tema	**Animaciones de interfaz y microinteracciones con Framer Motion**
Duración sugerida	**2 a 3 horas**
Modalidad	**Laboratorio práctico sobre el proyecto integrador**
Requisitos previos	**MVP React de semana 8 y componente Card tipado preparado en Desarrollo Web**
Entregable	**Card animada con entrada, hover/tap y al menos una microinteracción accesible**

2. Propósito del laboratorio

Mejorar la experiencia de interacción de la Plataforma Web de Experiencias Interactivas mediante animaciones intencionales, fluidas y consistentes. La animación debe orientar al usuario y ofrecer retroalimentación, no distraerlo ni dificultar el uso de la interfaz.

3. Resultados de aprendizaje

- Instalar y configurar Framer Motion (paquete motion) en una aplicación React.
- Aplicar animaciones de entrada, hover y tap a una tarjeta de experiencia.
- Organizar estados de animación mediante variants y transiciones reutilizables.
- Incorporar movimiento asociado al desplazamiento cuando tenga sentido para la interfaz.
- Verificar teclado, foco, preferencia de movimiento reducido y funcionamiento en móvil.

4. Escenario de trabajo

El catálogo de experiencias ya muestra datos mediante una Card tipada. Ahora se debe mejorar la respuesta visual: al aparecer en pantalla, la tarjeta entra suavemente; al pasar el puntero o enfocarla, ofrece una señal discreta; al pulsar el control, responde de forma inmediata. El resultado debe sentirse profesional y mantener el contenido legible.

5. Preparación

1. Abre el proyecto integrado y crea una rama de trabajo, por ejemplo immersive-lab.
2. Comprueba que la versión base funcione y registra el estado inicial.
3. Instala el paquete actual: npm install motion.
4. Verifica la versión de React y que las dependencias se instalen sin conflictos.
5. Importa los elementos desde motion/react, siguiendo la documentación de la versión instalada.

Nota de compatibilidad: en proyectos existentes con framer-motion, sigue la convención y versión que ya use el repositorio; no migres paquetes sin necesidad. Esta guía utiliza la API actual del paquete motion.

6. Experimento 1: animación de entrada

En el componente de tarjeta, reemplaza el contenedor principal por motion.article. Define un estado inicial y uno visible.

```
import { motion } from "motion/react";

<motion.article
  initial={{ opacity: 0, y: 18 }}
  animate={{ opacity: 1, y: 0 }}
  transition={{ duration: 0.35, ease: "easeOut" }}
  className="experience-card"
>
  {/* contenido existente de la tarjeta */}
</motion.article>
```

Observa: ¿la entrada ayuda a reconocer la aparición de contenido? Ajusta duración y desplazamiento con moderación. Evita animaciones largas que retrasen el acceso a la información.

7. Experimento 2: hover y tap

Agrega retroalimentación visual discreta al contenedor. Si la tarjeta contiene controles interactivos, no hagas que el efecto visual interfiera con ellos.

```
<motion.article
  whileHover={{ y: -4, scale: 1.01 }}
  transition={{ type: "spring", stiffness: 320, damping: 24 }}
  className="experience-card"
>
  {/* contenido */}
</motion.article>

<motion.button
  type="button"
  whileTap={{ scale: 0.97 }}
  onClick={handleSelect}
>
  Ver experiencia
</motion.button>
```

No uses hover como única señal de interacción: en pantallas táctiles no existe el puntero. Mantén texto, contraste y estados de foco visibles.

8. Experimento 3: variants reutilizables

Centraliza los estados para facilitar el mantenimiento y lograr una experiencia consistente en todo el catálogo.

```
const cardVariants = {
  hidden: { opacity: 0, y: 16 },
  visible: {
    opacity: 1,
```

```
    y: 0,
    transition: { duration: 0.3, ease: "easeOut" }
  }
};

<motion.article
  variants={cardVariants}
  initial="hidden"
  animate="visible"
  whileHover={{ y: -3 }}
>
  {/* contenido de la tarjeta */}
</motion.article>
```

9. Experimento 4: animación al entrar en pantalla

Para un catálogo largo, evita animar todos los elementos desde el primer render si están fuera de la vista. Puedes activar la animación cuando la tarjeta entre en el área visible.

```
<motion.article
  initial={{ opacity: 0, y: 14 }}
  whileInView={{ opacity: 1, y: 0 }}
  viewport={{ once: true, amount: 0.2 }}
>
  {/* contenido */}
</motion.article>
```

Comprueba el comportamiento al desplazarte hacia abajo y al volver a subir. Usa este recurso solo cuando mejore la orientación o comprensión del contenido.

10. Experimento 5: arrastre (drag)

Prueba el arrastre en un elemento secundario claramente identificable, como una ficha de exploración. No conviertas una tarjeta completa en un objeto arrastrable si eso dificulta el desplazamiento vertical en móviles.

```
<motion.div
  drag
  dragConstraints={{ left: -80, right: 80, top: -20, bottom: 20 }}
  whileDrag={{ scale: 1.03 }}
>
  Explora esta ficha
</motion.div>
```

El arrastre es opcional en el reto final. Si no aporta valor real al usuario, documenta por qué decidiste no incorporarlo.

11. Accesibilidad y movimiento reducido

- Comprueba que todos los controles puedan operarse con teclado y que el foco sea visible.
- No comuniques información esencial únicamente mediante movimiento, color o posición.
- Respeta la preferencia del sistema para reducir movimiento. Motion ofrece useReducedMotion para adaptar efectos.
- Evita parpadeos, escalados excesivos y desplazamientos que puedan causar molestias.

- Verifica contraste, etiquetas accesibles y áreas de interacción suficientes en móvil.

```
import { useReducedMotion } from "motion/react";

const reduceMotion = useReducedMotion();

<motion.article
  initial={{ opacity: 0, y: reduceMotion ? 0 : 14 }}
  animate={{ opacity: 1, y: 0 }}
  transition={{ duration: reduceMotion ? 0 : 0.3 }}
>
  {/* contenido */}
</motion.article>
```

12. Reto integrador

Integra en el proyecto una Card de experiencia tipada proveniente de la guía de Desarrollo Web y aplica:

1. Una animación de entrada o de aparición en viewport.
2. Una respuesta hover moderada y una respuesta tap en un control.
3. Una transición consistente mediante variants o configuración equivalente.
4. Una alternativa respetuosa con la preferencia de movimiento reducido.
5. Pruebas de funcionamiento con mouse, teclado y pantalla táctil o emulación móvil.

13. Protocolo de pruebas

Prueba	Procedimiento	Criterio de aceptación	Cumple
Entrada	**Recargar la vista y observar la tarjeta**	**Aparece suavemente sin retrasar el uso**	☑
Hover	**Pasar el puntero sobre la tarjeta**	**Respuesta discreta y sin saltos de diseño**	☑
Tap	**Pulsar el botón principal**	**Retroalimentación breve; acción funcional**	☑
Viewport	**Desplazarse por el catálogo**	**Animación solo cuando aporta valor**	☑
Teclado	**Navegar con Tab y activar controles**	**Foco visible y operación posible**	☑
Movimiento reducido	**Activar preferencia del sistema o simularla**	**Se reduce o elimina movimiento no esencial**	☑
Responsive	**Probar vista móvil y escritorio**	**Sin desbordamiento ni bloqueos de scroll**	☑
Integración	**Usar datos reales de la Card tipada**	**No se rompe el contrato ni la funcionalidad**	☑

14. Evidencias y entrega

- Repositorio actualizado con instrucciones para ejecutar el proyecto.
- Código de la Card tipada y animada integrado en la aplicación.
- Video mostrando entrada, hover/tap, responsive y prueba de teclado.
- Captura o nota de la prueba de movimiento reducido.
- Reflexión breve: qué microinteracción aporta valor y qué efecto se descartó por ser innecesario.

15. Preguntas de análisis

1. ¿Qué intención de usuario comunica la animación de entrada?
2. ¿Qué diferencia hay entre una animación decorativa y una microinteracción útil?
3. ¿Por qué hover no es suficiente para dispositivos táctiles?
4. ¿Qué problemas puede generar animar demasiados elementos al desplazarse?
5. ¿Cómo se relaciona una Card tipada con una Card animada dentro del proyecto compartido?

16. Cierre

El laboratorio no consiste en añadir movimiento por decoración. El equipo debe demostrar que la animación mejora la retroalimentación y la percepción de calidad, preservando rendimiento, accesibilidad y claridad. El producto de la semana es la misma Card del proyecto integrador: robusta por TypeScript y expresiva mediante Framer Motion.

Referencias técnicas

- **Motion for React — documentación: https://motion.dev/docs/react**
- **Motion — Accessibility: https://motion.dev/docs/react-accessibility**
- **React — TypeScript: https://react.dev/learn/typescript**

Anexo de la entrega

Laboratorio de Experiencias Web Inmersivas · Semana 9 · Wayra Academy

Resultados del protocolo de pruebas

Entrada	La tarjeta aparece en 317 ms, con 17 cuadros intermedios
Hover	Sube 3 px, escala 1, sin saltos de diseño; vuelve a 0 al salir
Tap	Escala 0,97 al pulsar y 1 al soltar; abre el temario y fija `?curso=react`
Viewport	2 tarjetas visibles al cargar y 14 esperando; las 16 aparecen al recorrer; no se repite al volver
Teclado	24 paradas con Tab, todas con anillo de foco; ninguna en una tarjeta invisible; Enter activa los botones
Movimiento reducido	Las 16 visibles sin desplazarse; sin transformaciones en hover, tap ni estrella; el favorito funciona
Responsive	Sin scroll horizontal en 1280, 768 y 375 px; deslizar sobre la tarjeta mueve unos 205 px, igual que el control
Integración	16 tarjetas con datos reales; filtros, búsqueda, favoritos y `?curso=id` funcionan; 5 rutas sin errores en consola

El táctil y el movimiento reducido se emularon en Chrome. No se probaron en un teléfono físico ni con un lector de pantalla real.

Prueba de movimiento reducido

Se activó la preferencia `prefers-reduced-motion: reduce` y se recargó el explorador. Las tarjetas se mostraron de inmediato y no hubo desplazamiento en hover ni escala al pulsar.

Reflexión breve

La microinteracción que más valor aporta es la respuesta al pulsar los botones junto con el pulso de la estrella del favorito. Confirma de inmediato que la acción se registró, y eso es útil sobre todo en móvil, donde no existe el hover. Además, no se depende de ella: la etiqueta y `aria-pressed` siguen comunicando el estado aunque se desactive el movimiento.

El efecto descartado fue el arrastre. Era vistoso, pero el catálogo no tiene nada que ordenar y podría estorbar el desplazamiento en móvil. También se quitó el `scale` de la tarjeta por la nitidez del texto. Las pruebas enseñaron algo más: el peso importa, porque Motion completo aumentaba el JavaScript en 47 %, y el hover pegado en táctil solo se descubrió al mirar una captura.

Preguntas de análisis

1. **¿Qué intención de usuario comunica la animación de entrada?** Indica que el contenido acaba de llegar y dónde mirar. Tras la carga, marca el paso de los esqueletos a los cursos reales.

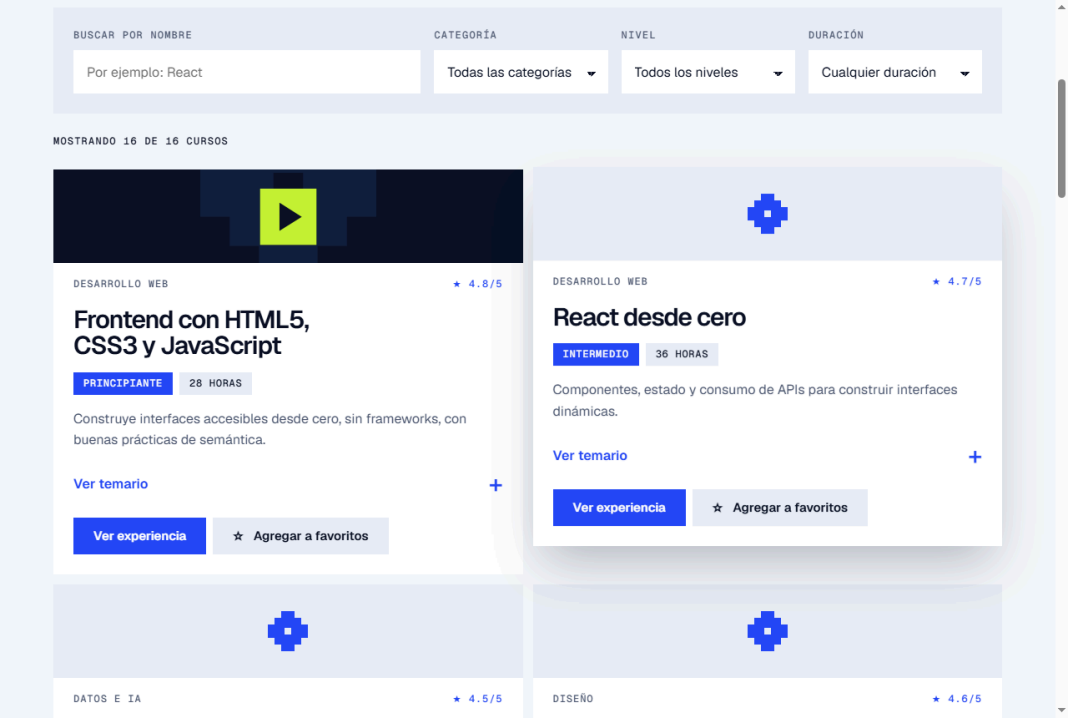
2. **¿Qué diferencia hay entre una animación decorativa y una microinteracción útil?** La decorativa solo adorna; si se quita, nada cambia. La útil responde a una acción y da información, como confirmar que un botón se pulsó.

3. **¿Por qué hover no es suficiente en dispositivos táctiles?** Porque no hay puntero. Además, el `:hover` de CSS puede quedarse pegado tras un toque, como pasó aquí con el botón de favorito. Por eso hay respuesta al pulsar y estados con texto.

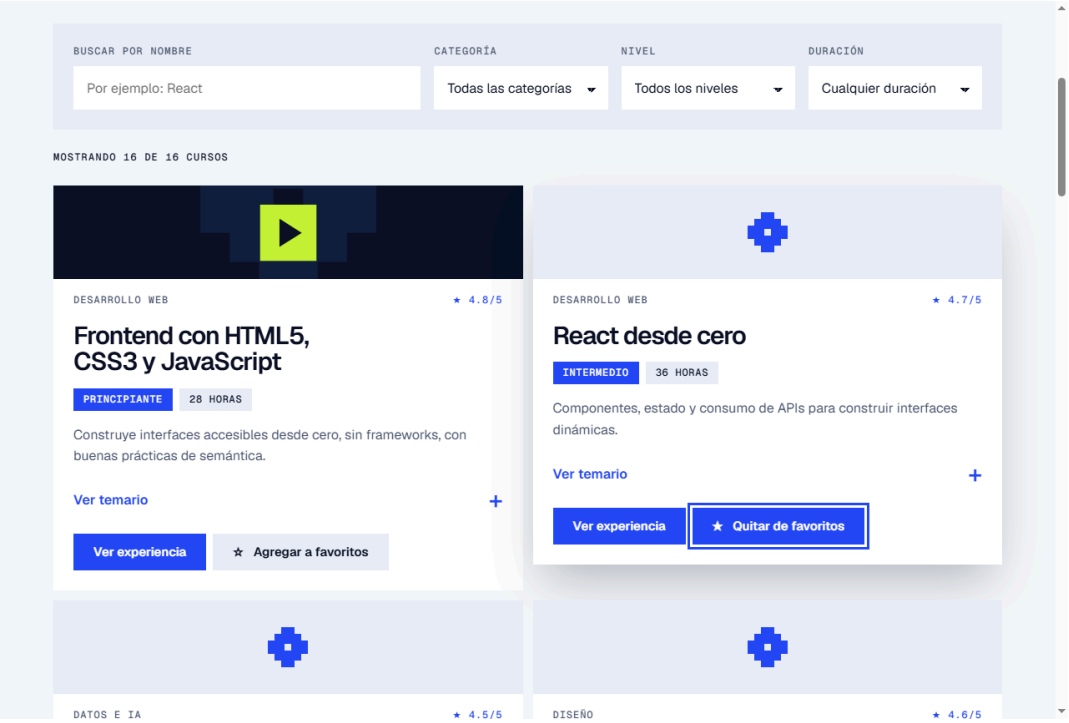
4. **¿Qué problemas puede generar animar demasiados elementos al desplazarse?** Distrae, retrasa la lectura, consume CPU y batería, y puede marear. Por eso cada tarjeta se anima una sola vez, con `once: true`, y no se repite al volver.

5. **¿Cómo se relaciona la Card tipada con la Card animada?** Es el mismo componente. TypeScript garantiza los datos y Motion añade la respuesta visual sin cambiar `Experience` ni las props. Las animaciones también están tipadas, con `Variants`.

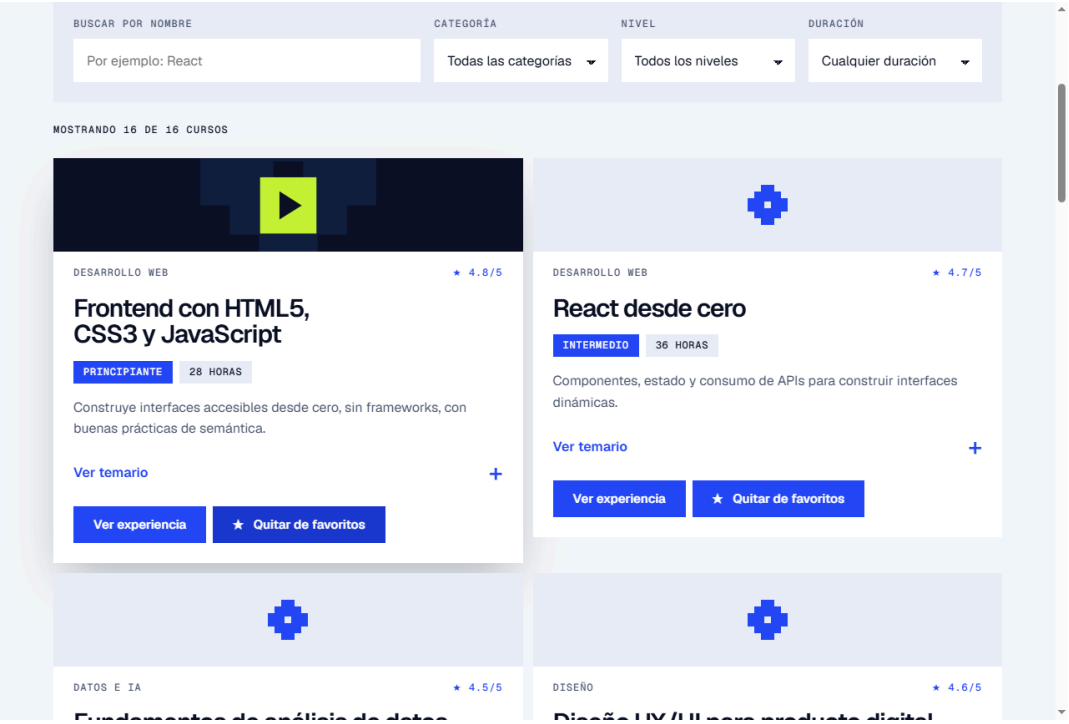
Capturas



Hover: la tarjeta sube 3 px y gana sombra, sin mover el resto del diseño.



Teclado: anillo de foco visible y tarjeta señalada; Enter activó el favorito.



Movimiento reducido: tarjetas visibles de inmediato y sin desplazamiento.



Móvil con toque emulado: el botón vuelve a su estado normal tras desmarcar.